SALUNIC - Sistema de Salud y Bienestar

IX. Desarrollo del prototipo en Python

Proyecto:	SALUNIC (App Tkinter + Modulo Consola integrados)
Python 3	**Leng**
Tkinter	**Interf**
https://github.com/chrissaraviagomez-lab/salunic-app	**Repo**

Documento de entrega que describe los modulos implementados, la validacion de datos y el manejo de excepciones, las operaciones CRUD, los algoritmos de busqueda y ordenamiento, y la interfaz en Tkinter, con espacios para capturas de pantalla y el enlace al repositorio.

1. Modulos implementados

El prototipo se organizo con una arquitectura modular para separar modelos, datos, logica e interfaces, cumpliendo la estructura requerida (Entidades, Datos, Funciones) y manteniendo integrados el modulo por consola y la aplicacion grafica.

1.1 Entidades/ (modelos - DataClass)

Contiene las 4 clases de datos del sistema, implementadas con @dataclass (dataclasses de Python):

- usuario.py -> clase Usuario (id, nombre, email, password, telefono)
- cita_medica.py -> clase CitaMedica (id, paciente_id, medico_id, especialidad, fecha, hora, lugar)
- medicamento.py -> clase Medicamento (id, nombre, dosis, frecuencia, presentacion)
- almacen_medicamento.py -> clase AlmacenMedicamento (id, medicamento_id, cantidad, fecha_vencimiento)
- __init__.py -> exporta las entidades para importacion centralizada.

1.2 Datos/ (almacenamiento en memoria y persistencia)

Gestiona las listas de objetos y el archivo JSON compartido:

- datos.py: define listas globales (usuarios, citas, medicamentos, almacen), datos iniciales de prueba, y funciones cargar_datos_iniciales(), guardar_datos() y cargar_datos().
- Persistencia en salunic_datos.json (raiz del proyecto), compartido entre app y consola.
- Serializacion con dataclasses.asdict() y reconstruccion con Clase(**dict).

1.3 Funciones/ (logica de negocio, una por DataClass + utilidades)

Implementa la logica del sistema. Regla aplicada: Funciones solo usa Datos y Entidades.

- usuario.py, cita_medica.py, medicamento.py, almacen_medicamento.py: operaciones CRUD y consultas por cada entidad.
- validaciones.py: validacion de entradas y manejo de errores de datos (ValueError).
- busqueda_lineal.py: algoritmos de busqueda lineal.
- cola_atencion.py: cola de atencion con prioridad usando heapq.
- sistema.py: servicios de alto nivel para la app (iniciar/guardar, autenticacion, registro, citas, cola) sin exponer Datos directamente a la UI.

1.4 app/ (interfaz grafica Tkinter)

Aplicacion de escritorio con navegacion entre pantallas. Regla aplicada: screens/ solo usa Funciones y Entidades.

- app/main.py: inicializa la app, precarga datos (Funciones.sistema.iniciar_sistema) y gestiona el cambio de pantallas (frames).
- app/screens/: pantallas como Login, Registro, Home, Citas, Panel de Atencion, Historial, Medicamentos, Estadisticas y formularios.
- app/data/users_data.py: adaptador de compatibilidad para operaciones de usuarios sobre la estructura unificada.
- app/screens/styles.py: estilos/colores y utilidades visuales (gradientes, componentes).

1.5 consola/ (modulo por consola)

Interfaz interactiva por menus para demostrar la logica requerida usando la misma estructura:

- consola/main.py: menu principal y submenus (Usuarios, Citas, Medicamentos, Almacen/Inventario, Cola de Atencion con Prioridad, Archivo de Datos). Usa Entidades, Datos y Funciones.

1.6 Otros modulos/carpetas de apoyo

- core/: capa de compatibilidad que delega a Entidades/Datos/Funciones (estado unificado), conservada para no romper referencias previas.
- docs/pdfs/: documentacion y guias/PDFs de estudio y entrega.
- scripts/: scripts auxiliares (generacion de PDFs).
- salunic_datos.json: archivo de persistencia compartido.

2. Validacion de datos y manejo de excepciones

Se implementaron validaciones para asegurar la calidad de los datos ingresados (especialmente en consola y en operaciones de registro/edicion) y manejo de excepciones para evitar fallos ante entradas invalidas o errores de E/S.

2.1 Validacion de datos (Funciones/validaciones.py)

Funciones implementadas (lanzan ValueError cuando el dato no cumple):

- validar_nombre(texto): no vacio; solo letras/espacios (incluye acentos); normaliza formato (title).
- validar_email(email): no vacio; formato valido (expresion regular usuario@dominio).
- validar_password(password): minimo 4 caracteres.
- validar_fecha(fecha): formato DD/MM/AAAA con fecha real (datetime.strptime).
- validar_hora(hora): formato HH:MM valido (datetime.strptime).
- validar_dosis(dosis): no vacia.
- validar_celular(texto): no vacio; digitos/simbolos permitidos (+, -, espacio), longitud controlada.
- validar_cantidad(cantidad): no vacia; entero positivo (isdigit / conversion).
- validar_descripcion(texto): no vacia; normaliza texto.
- pedir_campo(etiqueta, validador): bucle interactivo que reintenta hasta recibir un valor valido, capturando ValueError y mostrando el mensaje de error.

2.2 Manejo de excepciones

Se utilizan bloques try/except en puntos criticos:

- Entrada de datos en consola: pedir_opcion() valida que la opcion sea numerica; pedir_campo() captura ValueError por validaciones.
- Persistencia JSON (Datos/datos.py y capas relacionadas): try/except al guardar y cargar (errores de archivo, formato JSON, decodificacion); se informa al usuario con mensajes [OK]/[ERROR]/[INFO].
- Interfaz Tkinter (app): validaciones basicas en formularios (campos vacios, formato de correo, longitud de contrasena) con messagebox.showerror; y try/except en acciones que interactuan con datos/sistema para evitar cierres inesperados (ej. carga inicial, agregar cita/cola).
- Reglas de negocio: se previenen duplicados (ej. usuario por email) y se retornan valores/estados seguros (None/False) cuando corresponde.

```
# Ejemplo de validacion + manejo de excepcion (consola/Funciones)
def pedir_campo(etiqueta, validador):
    while True:
        try:
            valor = input(etiqueta)
            return validador(valor)      # puede lanzar ValueError
        except ValueError as e:
            print(f"  [Error] {e}")      # manejo controlado y reintento
```

3. Operaciones CRUD

Se implementaron operaciones CRUD (Crear, Leer/Buscar/Listar, Actualizar, Eliminar) para las entidades principales, organizadas en un archivo de funciones por cada DataClass dentro de Funciones/. Ademas se incluyen conteos/listados y operaciones sobre el inventario (almacen/stock).

3.1 Usuario (Funciones/usuario.py)

- Crear: registrar_usuario() - solicita datos, valida, verifica duplicado por email, asigna ID y agrega a la lista.
- Leer/Buscar: buscar_usuario(email), listar_usuarios().
- Actualizar: actualizar_usuario(email) - permite modificar campos (con validacion).
- Eliminar: eliminar_usuario(email).
- Conteo: contar_usuarios().

3.2 Cita Medica (Funciones/cita_medica.py)

- Crear: registrar_cita() - registra cita con paciente, medico, especialidad, fecha, hora y lugar (validaciones de fecha/hora).
- Leer/Buscar: buscar_cita(medico_id), listar_citas(lista=None).
- Actualizar: actualizar_cita(medico_id).
- Eliminar: eliminar_cita(medico_id).
- Conteo: contar_citas().

3.3 Medicamento (Funciones/medicamento.py)

- Crear: registrar_medicamento().
- Leer/Buscar: buscar_medicamento(nombre), listar_medicamentos(lista=None).
- Actualizar: actualizar_medicamento(nombre).
- Eliminar: eliminar_medicamento(nombre).
- Conteo: contar_medicamentos().

3.4 Almacen / Inventario (Funciones/almacen_medicamento.py)

- Crear: registrar_stock() (asocia stock a medicamento_id, cantidad, vencimiento).
- Leer/Buscar: buscar_stock(medicamento_id), listar_stock(lista=None).
- Actualizar: actualizar_stock(medicamento_id).
- Eliminar: eliminar_stock(medicamento_id).
- Conteo: contar_stock().

3.5 Integracion de CRUD con la app y consola

- Consola: submenus invocan directamente las funciones CRUD por entidad y permiten guardar/cargar JSON.
- App: utiliza Funciones.sistema y funciones asociadas para operaciones como autenticacion/registro, consulta y agregado de citas, respetando la regla de acceso (screens no accede directo a Datos).

4. Algoritmos de busqueda y ordenamiento

4.1 Busqueda lineal (Funciones/busqueda_lineal.py)

Se implemento busqueda lineal (secuencial), que recorre la lista elemento por elemento comparando el criterio hasta encontrar coincidencia(s). Es clara, didactica y adecuada para las estructuras en memoria del prototipo.

```
def busqueda_lineal(lista, criterio, atributo):
    resultados = []
    for elemento in lista:                          # recorrido secuencial
        valor = getattr(elemento, atributo, None)
        if isinstance(valor, str) and isinstance(criterio, str):
            if valor.lower() == criterio.lower():
                resultados.append(elemento)
        elif valor == criterio:
            resultados.append(elemento)
    return resultados
```

- busqueda_lineal_por_email(usuarios, email): busqueda especifica usada para localizar usuarios (y en la autenticacion/login via la logica del sistema).

- busqueda_lineal_citas_por_medico(citas, medico_id) y busqueda_lineal_medicamento_por_nombre(medicamentos, nombre): variantes de filtrado lineal.

4.2 Ordenamiento / atencion por prioridad (heapq)

Para la gestion de atencion se implemento una cola con PRIORIDAD (no FIFO puro), utilizando el modulo heapq de Python. El ordenamiento/seleccion del siguiente elemento se basa en una clave de prioridad:

- Adulto mayor: prioridad 0 (mayor prioridad).

- Normal: prioridad 1.

- turno_global: contador incremental para desempate (mantiene orden de llegada dentro del mismo nivel de prioridad).

- Estructura en heap: tuplas (prioridad, turno_global, item); heapq.heappop() extrae siempre la tupla de menor prioridad/turno, por lo que el adulto mayor se atiende primero aunque haya llegado despues.

```
import heapq
# encolar
prioridad = 0 if es_adulto_mayor else 1
heapq.heappush(self.cola, (prioridad, self.turno_global, item))
# atender (ordenado por prioridad)
prioridad, turno, atendido = heapq.heappop(self.cola)
```

Adicionalmente, para mostrar la cola en pantalla/consola se presenta una vista ordenada (sorted) por (prioridad, turno) sin alterar la logica de extraccion del heap.

5. Interfaz en Tkinter

La interfaz grafica se desarrollo con Tkinter (app/). La app utiliza un contenedor principal y muestra pantallas (Frames) segun la navegacion. El diseno visual mantiene un estilo consistente (fondos/gradientes, tarjetas, colores, tipografia) definido en app/screens/styles.py.

5.1 Estructura y navegacion

- app/main.py (clase SalunicApp): crea la ventana (375x812, no redimensionable), contenedor de pantallas y metodo show_screen() para cambiar de vista.
- Pantallas principales: Splash/Inicio, Login, Registro, Home, Citas, Panel de Atencion, Historial, Medicamentos, Estadisticas y formularios (ej. FormMedicamento, FormRegistro, flujo OTP de recuperacion).
- Carga inicial de datos: Funciones.sistema.iniciar_sistema() al iniciar, para trabajar con la persistencia compartida.

5.2 Funcionalidades visibles en la GUI

- Login: validacion de campos y autenticacion mediante la logica del sistema (busqueda lineal por email en la capa de Funciones/Datos).
- Registro de usuario: crea el usuario (DataClass) a traves de Funciones.sistema y guarda en JSON.
- Citas: visualiza citas desde la logica compartida, permite agregar nuevas citas y enviar pacientes a la cola indicando si es adulto mayor (prioridad).
- Panel de Atencion: muestra el paciente "EN ATENCION" y la "COLA DE ESPERA" ordenada por prioridad; boton "ATENDER SIGUIENTE" ejecuta el desencolado por prioridad (heapq).

5.3 Capturas de pantalla (insertar en los espacios indicados)

[CAPTURA DE PANTALLA] 1. Pantalla de inicio/login

Insertar aqui la captura: Ventana principal de la app Tkinter mostrando acceso/login (y/o splash/inicio).

[CAPTURA DE PANTALLA] 2. Registro / Home

Insertar aqui la captura: Formulario de registro o pantalla Home con opciones del sistema.

[CAPTURA DE PANTALLA] 3. Gestion de Citas

Insertar aqui la captura: Pantalla de Citas: listado y opcion para agregar nueva cita / agregar a cola con prioridad.

[CAPTURA DE PANTALLA] 4. Panel de Atencion (cola con prioridad)

Insertar aqui la captura: Panel mostrando EN ATENCION y COLA DE ESPERA; evidencia de prioridad (adulto mayor primero) y boton Atender Siguiente.

[CAPTURA DE PANTALLA] 5. Modulo por consola

Insertar aqui la captura: Ejecucion de py consola/main.py: menu principal y/o submenu de Cola de Atencion con Prioridad y Archivo de Datos (JSON).

Nota: inserte las capturas reales tomadas durante la ejecucion de la app (py app/main.py) y la consola (py consola/main.py). Las imagenes deben evidenciar el funcionamiento de login/registro, CRUD/citas, cola con prioridad, y persistencia JSON.

6. Enlace al repositorio

El codigo fuente del prototipo se encuentra en el siguiente repositorio:

https://github.com/chrissaraviagomez-lab/salunic-app

Contenido relevante en el repositorio: carpetas Entidades/, Datos/, Funciones/, app/, consola/, docs/pdfs/, scripts/, archivo salunic_datos.json y README.md con instrucciones de ejecucion y estructura.

Como ejecutar (referencia)

```
# Aplicacion grafica (Tkinter)
py app/main.py

# Modulo por consola
py consola/main.py
```

Resumen de cumplimiento de la indicacion IX

- Modulos implementados: descritos por carpetas/responsabilidades (Entidades, Datos, Funciones, app, consola, core compatibilidad).
- Validacion de datos y manejo de excepciones: validadores (ValueError), reintentos en consola, try/except en JSON e interfaz.
- Operaciones CRUD: implementadas por entidad (Usuario, CitaMedica, Medicamento, Almacen/Stock).
- Algoritmos de busqueda y ordenamiento: busqueda lineal; cola con prioridad/orden por prioridad mediante heapq (turno de desempate).
- Interfaz en Tkinter: navegacion por pantallas, login/registro/citas/panel de atencion.
- Capturas de pantalla: espacios indicados para insercion de evidencias.
- Enlace al repositorio: https://github.com/chrissaraviagomez-lab/salunic-app